Universidad de Jaén
Departamento de Informática

Actividades Tema 4

Configuración y Despliegue de Aplicaciones en la Nube

Octubre, 2026

v 1.0

Contenidos

Introducción **1**
Descripción de las actividades propuestas **2**
Actividad 4.1.- Diseño y despliegue de una aplicación en la nube con Kubernetes. 2
Preparación 2
Tarea 1.- Configuración inicial de un despliegue local K8s 4
Tarea 2.- Análisis estático de seguridad del despliegue 4
Tarea 3.- Ajuste de recursos y configuración dinámica de Pods 5
Actividad 4.2.- Segmentación de la red y despliegue de un clúster kubernetes en la nube 7
Tarea 1.- Segmentación de tráfico de red en un clúster K8s 7
Tarea 2.- Despliegue en clúster K8s gestionado en la nube 9

Introducción

Uno de los aspectos clave tratados en el tema 4 ha sido la importancia de diseñar arquitecturas de aplicaciones distribuidas de forma declarativa o IaaC para facilitar el análisis y seguimiento de los cambios o vulnerabilidades en las configuraciones de despliegue. Otro de los aspectos ha sido el uso de plataformas que nos permitan abstraer los recursos específicos proporcionados por los proveedores, además de disponer de los elementos básicos necesarios para el uso de patrones habituales en este tipo de arquitecturas. Kubernetes es una de las plataformas de orquestación que más difusión

están teniendo en este sentido y es una pieza clave a la hora de diseñar la arquitectura segura de una aplicación en la nube garantizando su alta disponibilidad y tolerancia a fallos.

En estas actividades se propone complementar el estudio de algunas de las facilidades proporcionadas por esta plataforma y familiarizarse con ella realizando algún despliegue sencillo donde se apliquen las técnicas estudiadas en el tema 4. Cuando nuestro despliegue esté operativo, procederemos a desplegarlo en un clúster k8s en la nube gestionado por el servicio Google Kubernetes Engine, GKS.

Descripción de las actividades propuestas

Actividad 4.1.- Diseño y despliegue de una aplicación en la nube con Kubernetes.

En esta actividad crearemos un despliegue simple de la aplicación de pruebas diseñada en las actividades del tema 3 y lo lanzaremos en primer lugar en un clúster local para experimentación, e.g. Docker Kubernetes System, DKS, basado en *kind*, que es la **opción recomendada**, o creado directamente con las utilidades *kind* o *minikube*. Para ello, comenzaremos realizando algunas pruebas en nuestro ordenador de trabajo[1] con una aplicación simple para familiarizarnos con las operaciones básicas con kubernetes y comprobar que tenemos correctamente instaladas las **herramientas necesarias**, descritas en el documento de software de la asignatura: instalación *docker* con *clúster local k8s, kubectl, helm y utilidad de monitorización de clústers k9s.*

Preparación

En este apartado veremos cómo lanzar en un el despliegue de una aplicación básica en un clúster k8s formada por un servicio web de dos pods con el contenedor *traefik/whoami,* usado también en las actividades del tema 3. Para el acceso al servicio, se utilizará un servicio *ingress* que se encargará del enrutamiento de las peticiones externas hacia el servicio situado en el interior del clúster. *IMPORTANTE: esta parte de la actividad no debe incluirse en el informe; sirve únicamente para verificar que funciona correctamente el clúster local K8s y familiarizarse con los conceptos y operaciones básicas de uso de K8s.*

Pasos:

1. **Descargarse el repositorio** de ejemplos de la asignatura https://github.com/jrbalsas/asnwebapps. IMPORTANTE: si ya se hubiera descargado previamente, sincronizar la última versión (*git pull*) por si hubiera nuevos cambios en el repositorio remoto.

[1] En caso de no poder instalar ningún clúster local como se describe en el documento de software de la asignatura, se puede hacer uso de la instalación de *kind* o *minikube* en cloud-shell

2. **Verificar que hay conexión** con el clúster kubernetes local con el comando "*kubectl cluster-info"*. Si no hubiera conexión, comprobar que k8s está lanzado correctamente.
3. Instalar el **sevicio de métricas** en el contenedor para poder consultar el uso de memoria y cpu de los diferentes elementos desplegados. Accederemos a la carpeta *k8s/metrics-server* y utilizaremos la herramienta *helm* que debe haberse instalado previamente. Ejecutaremos el script *./helm_install.sh* que en cargará de realizar los pasos necesarios. Consultar en el fichero los detalles de uso de dicha herramienta muy utilizada para simplificar la instalación de recursos habituales en k8s.
4. **Instalar un controlador ingress** en el clúster para poder utilizar recursos de este tipo. Accederemos a la carpeta *k8s/ingress-controller/traefik* y utilizaremos la herramienta *helm* para instalar traefik[2] como controlador de recursos ingress. Ejecutaremos el script *./helm_install.sh* que se encargará de realizar los pasos necesarios y que es conveniente revisar para conocer los detalles de uso.
5. Acceder desde un terminal a la carpeta *k8s/testapp y* **lanzar el despliegue de la aplicación de prueba** con el comando "*kubectl apply -f .*"
6. **Comprobar el estado de los diferentes recursos**, e.g. usando alguno de los siguientes comandos "*kubectl get all*", "*kubectl get services*", "*kubectl get pods*", etc. También se puede utilizar *vscode* con las extensiones de kubernetes, google cloud o lanzar en otro terminal la utilidad *k9s* que permitirá consultar fácilmente toda la información del contenedor.
7. Una vez que los recursos estén creados, **acceder a la aplicación** desde el navegador en la url http://localhost/[3] o http://localhost/?env=true para que el contenedor muestre también las variables de entorno que recibe del clúster.
8. Si algún recurso no se lanza correctamente, puede consultarse su estado con el comando "*kubectl describe tipo recurso*", e.g. *kubectl describe deployment testapp*
9. **Estudiar el contenido de los ficheros** testapp.yaml y lb-ingress.yaml para comprender la estructura del despliegue de la aplicación.
10. Si fuera necesario, experimentar con algún cambio en los ficheros de despliegue, por ejemplo, para aumentar el número de réplicas de los pods del servicio, solo hay que editarlo y aplicarlo volviendo a utilizar el comando "*kubectl apply -f ficheromodificado.yaml*".
11. Una vez que se haya terminado de trabajar con el despliegue, se pueden eliminar todos los recursos con el comando "*kubectl delete -f .*" desde la carpeta donde se encuentran los ficheros de configuración de la aplicación.

Se pueden consultar más detalles de uso y ejemplos en las diapositivas del tema 4 y en la documentación del proyecto K8s tal y como aparece en la bibliografía del mismo tema.

---

[2] Traefik tienen habilitada la consola de consulta de estado del balanceador que estará disponible en la url /dashboard/ (*la barra final es obligatoria*)

[3] Si el clúster se ha creado en linux utilizando manualmente la herramienta *kind*, consultar la ip de cualquiera de los nodos del clúster, `kubectl get nodes -o wide`, e.g. 172.18.0.3, y consultar donde se ha asignado en el nodo el puerto del servicio ingress, `kubectl get svc -n traefik`, `e.g. 80:30786`, con lo que accedemos en la url `http://172.18.0.3:30786`. Con *minikube* debe consultarse la url pública donde se expone el servicio usando el comando `minikube service list`

Una vez realizada la experimentación con un despliegue en K8s, se realizarán las siguientes **tareas para completar la actividad**:

Tarea 1.- Configuración inicial de un despliegue local K8s

Acceder a la carpeta *phpwebappv2*[4] utilizada en la actividad 3.2 y completar su configuración de despliegue en K8s creando en el subdirectorio k8s los siguientes ficheros:

- *wwwserver.yaml* donde se defina adecuadamente un *deployment* para lanzar 2 réplicas del contenedor asn/phpwebapp:v1[5] y un servicio para dar acceso a dichas réplicas.
    - Es importante indicar la versión de la imagen o que la política de despliegue, *imagePullPolicy*, sea de tipo *IfNotPresent*, para que el clúster k8s local trate de obtener en primer lugar la imagen a desplegar de las disponibles en repositorio local de docker[6]. En otro caso trataría de hacerlo en primer lugar del repositorio público de docker hub en cuyo caso daría error al no encontrarse en el mismo.
    - Se deben proporcionar al despliegue las variables de entorno adecuadas con el usuario, clave y cadena de conexión a la base de datos para que la aplicación pueda conectarse al servidor de bases de datos[7].

- *loadbalancer.yaml*, que configure un recurso ingress para dar acceso al anterior servicio

Nota: el directorio ya incluye un fichero mariadb.yaml para desplegar un pod con una BD mariadb y el servicio correspondiente para que los pods con la aplicación web puedan acceder al mismo.

Lanzar el despliegue completo y comprobar que la aplicación responde adecuadamente accediendo mediante un navegador y observando cómo responden los frontales[8]. Consultar el estado de traefik accediendo a la url /dashboard/ (es importante usar la barra final). Aportar como evidencias capturas que muestren los recursos desplegados en el cluster, la aplicación web respondiendo desde el navegador y su url de acceso

Tarea 2.- Análisis estático de seguridad del despliegue

Una de las ventajas que aportan los despliegues basados en IaaC es que permiten utilizar analizadores estáticos que procesen los ficheros de configuración para detectar problemas en la configuración, malas prácticas o posibles vulnerabilidades realizadas durante el diseño

[4] La aplicación está disponible en el repositorio de ejemplos de prueba de la asignatura. https://github.com/jrbalsas/asnwebapps

[5] Esta imagen se generó durante la actividad 3.2, pero puede volver a generarse si es necesario, e.g. con el comando "*docker compose build .*"

[6] Si se ha creado un clúster *kind* de forma manual, es necesario cargar la imagen del contenedor construída en la máquina local dentro del clúster con el comando *kind load docker-image nombre_imagen*

[7] La aplicación debe recuperar dichos datos de configuración desde las variables de entorno como se estudió en la actividad 3.1

[8] Antes de acceder a la aplicación desde el navegador debe tenerse en cuenta que el pod con la BD MariaDB puede necesitar unos instantes hasta que realice las tareas de inicialización de la BD.

del despliegue. En esta ocasión utilizaremos el analizador estático, KICS[9], para revisar los ficheros de configuración de nuestro despliegue. KICS es una solución basada en software libre que recoje reglas desarrolladas por la comunidad para analizar ficheros de despliegue de multitud herramientas IaaC como K8s, Terraform, Docker compose, AWS CloudFormation, etc.
Realizar los siguientes pasos:

- **Ejecutar la KICS sobre el directorio que contiene los ficheros del despliegue** realizado en la tarea 1 siguiendo los pasos indicados la propia web de la herramienta. Es conveniente utilizar la opción *--report-formats "html"* para que el informe de resultados se genere en un fichero html que puede consultarse fácilmente desde el navegador. Al abrir dicho fichero, pulsar sobre los iconos de la parte superior para filtrar las vulnerabilidades encontradas según su nivel de riesgo.
- **Explicar brevemente las vulnerabilidades de nivel crítico y alto detectadas** y encontrar la forma correcta de **configurar el despliegue para evitar que aparezcan** dichas vulnerabilidades
- **Aplicar los cambios propuestos** al despliegue y comprobar que sigue operativo. Verificar que las vulnerabilidades en cuestión han desaparecido volviendo a hacer un análisis del despliegue.

Tarea 3.- Ajuste de recursos y configuración dinámica de Pods

En este apartado mejoraremos nuestro despliegue introduciendo ajustes para limitar los recursos que pueden utilizar los pods del clúster. También utilizaremos ficheros específicos donde definir los valores de las variables de entorno que se proporcionan a los pods al igual que hicimos en las actividades del tema 3. Realizar para ello los siguientes pasos:

1.- Revisar el último informe de la herramienta KICS generado en la tarea anterior y localizar **avisos de la categoría *"Resource Management"***. Explicar brevemente las vulnerabilidades de este tipo encontradas por en analizador y estudiar [1] cómo **evitarlas realizando los cambios oportunos en nuestro despliegue**.

Notas: tener en cuenta que en el despliegue en producción que haremos en la siguiente actividad dispondremos de recursos limitados y todo el despliegue no debería ocupar más de 1 GB de RAM. La memoria y cpu utilizada por los pods de un despliegue puede consultarse con el comando kubectl top pod/node aunque herramientas como k9s mostrarán información mucho más detallada

2.- Para evitar que los valores de configuración dinámica de los Pods estén en los ficheros de despliegue, haremos **uso del recurso *ConfigMap***. Consultar cómo crear en k8s un recurso de este tipo [1] que contenga las variables de entorno de nuestro despliegue y modificarlo para que los pods obtengan dichas variables desde el *ConfigMap* creado. Aplicar los cambios realizados y comprobar que el clúster sigue funcionando correctamente.

[9]Keeping Infraestructura as Code Secure, https://kics.io

Para la **entrega de la actividad** se proporcionará:

- Un **fichero comprimido con la carpeta de proyecto** para poder lanzar el despliegue local, que contendrá únicamente los diferentes ficheros necesarios: Dockerfile, ficheros .yaml con definición de recursos k8s, fuentes de la aplicación desplegada, ficheros de configuración adicionales necesarios, etc. para personalizar y lanzar los diferentes microservicios diseñados, así como el último informe generado por el analizador estático
- **Informe** con un resumen de los pasos realizados y capturas de pantalla donde muestre la consecución y correcto funcionamiento de las tareas planteadas.

Extensión aproximada del informe: 3-4 páginas (principalmente por las capturas de pantalla)
Tiempo estimado: 1-2 horas (considerando la consulta de información y pruebas)
(**plazo de entrega: 6 de noviembre**)

Material de trabajo

[1] Documentación de Kubernetes. Linux Foundation, 2026. [on-line] disponible en https://kubernetes.io/docs

- Las referencias concretas a consultar se pueden obtener de los enlaces proporcionados por el analizador estático utilizado en la tarea 2

Actividad 4.2.- Segmentación de la red y despliegue de un clúster kubernetes en la nube

Requisitos:
Para la realización de esta actividad es necesario al menos haber completado el despliegue descrito en la tarea 1 de la actividad 4.1. No obstante, se valorará positivamente partir de la versión mejorada del despliegue en las tareas 2 y 3 de la actividad correspondiente.

Tarea 1.- Segmentación de tráfico de red en un clúster K8s

Al igual que ocurre en una red física, la segmentación de tráfico entre los pods de un clúster k8s nos permitirá limitar las conexiones que pueden realizarse entre ellos. De esta forma, se dificultará a un potencial atacante que pueda realizar desplazamientos laterales entre pods del clúster si alguno de los pods resultara comprometido. A diferencia de los mecanismos de control de tráfico en redes físicas, en k8s las restricciones se pueden establecer fácilmente de forma centralizada sin tener que realizar configuraciones específicas en cada elemento físico de interconexión, como cortafuegos, routers, etc. Además, k8s permite aplicar rápidamente colecciones de reglas a cualquier pod identificándolo simplemente mediante etiquetas, por lo que normalmente no es necesario conocer las IPs específicas o subredes donde estos operan, lo que por otra parte sería complejo de mantener debido al carácter no permanente que tiene cualquier pod en un clúster de estas características.

En k8s, las restricciones de tráfico se establecen mediante recursos denominados *Network Policies*[10] que, como se ha indicado anteriormente, consisten en reglas de selección de pods (*podSelectors*) sobre los que se va a actuar y las restricciones que se establecerán para dichos pods en cuanto a tráfico entrante (*ingress*) o saliente (*egress*).

Estudiar la sintaxis básica de un recurso *NetworkPolicy* para limitar el tráfico entrante o saliente de un pod [2,3] y realizar los siguientes pasos:

1.- En un terminal independiente, **cree un Pod de pruebas en el clúster para verificar la efectividad de las reglas** a medida que se vayan aplicando al despliegue, e.g.

```
kubectl run --rm -it test-pod --image busybox -- sh
```
[11]

Utilice dicho pod para comprobar si tiene acceso a los puertos de los pods del despliegue a medida que haga cambios en la configuración. Por ejemplo, puede utilizar el siguiente comando en el pod de pruebas para comprobar (durante 2 segundos) si el puerto TCP de los pods de un servicio están abiertos o no :

[10] El clúster debe tener habilitado un plugin CNI (Container Network Interface) para que se puedan definir políticas de red. Los clústers basados en *kind, como el creado con Docker Desktop,* disponen de uno instalado por defecto, mientras que en clústers generados con *minikube* debe habilitarse de forma específica.
[11] *Busybox* es una aplicación que integra las funcionalidades de diferentes herramientas individuales como ping, curl, ip, ss, nc, tcpdump, etc. y se utiliza para evitar tener instalarlas de forma individual

```
# nc -zvw 2 testapp 80
```

2.- Revise los ejemplos de políticas de red de denegación de tráfico por defecto entre pods que hay en el directorio *k8s/network-policies* que permiten respectivamente: desactivar todo el tráfico entrante y saliente de los pods de un despliegue (pods marcados con etiquetas *app: asn-app*), permitir que los pods puedan consultar el servicio DNS interno del clúster para conocer la ip de cualquier servicio (aunque por ahora no puedan todavía establecer conexiones L4) y, por último, una regla específica que permite a un pod concreto (e.g. con etiqueta *run: testapp*) que pueda recibir conexiones entrantes por los puertos 80,443/TCP.

3.- Antes de crear las reglas para limitar el acceso a los Pods de nuestro despliegue, es necesario **identificar los pods de forma única atendiendo al papel que desempeñan** en el despliegue, e.g. frontales web, base de datos, etc. Para ello los etiquetaremos atendiendo a las limitaciones de tráfico que queremos aplicarles. Por ejemplo, vamos a utilizar una etiqueta *role* que determine el papel de cada pod en el despliegue, e.g. *role: wwwserver* para los nodos frontales y *role: database* para los pods de la base de datos.

Por lo tanto, edite los despliegues (*deployments*) de la base de datos y de los frontales web y añada al template de cada uno su etiqueta correspondiente (apartado template.metadata.labels), e.g. *role: database* y *role: wwwserver* respectivamente. Asegúrese de que estos pods también incluyen la etiqueta general para identificar cualquier pod de la aplicación, e.g. *app: asn-app*, que se utilizará más adelante para aplicarles las políticas de red por defecto estudiadas en el paso 1. Aplique los cambios en ambos despliegues que por ahora no afectarán al funcionamiento de la aplicación. Puede consultar las nuevas etiquetas asignadas en la descripción de cada despliegue o pod, e.g. *kubectrl get deployment/wwwserver -o wide* o usando k9s

4.- Cree una nueva regla específica (puede copiar una del paso inicial cambiando su nombre interno en metadata.name:) que permita **limitar el tráfico entrante en los pods frontales web**. Para ello, la regla solo debe permitir conexiones entrantes al puerto 80/TCP (omitimos el puerto 443 que no tenemos configurado y servirá para pruebas). Para que la regla se aplique solo a los pods con los frontales web de la aplicación, añada al selector de pods de la regla (*spec.podselector.matchLabels*) la etiqueta específica de dichos pods y que se les asignó anteriormente en sus ficheros de despliegue, e.g. *role: wwwserver*.
Aplique la regla creada y, si todo ha ido bien[12], la aplicación deberá seguir respondiendo desde el navegador. Sin embargo, si tratamos de verificar el puerto 443/TCP desde el pod de prueba comprobaremos que ahora estará cerrado para otros pods del clúster aunque el contenedor lo sigue teniendo abierto.

5.- El siguiente paso será **limitar el tráfico entrante del Pod de la base de datos**. Para ello crearemos una nueva regla que limite sus conexiones entrantes solo al puerto

[12] IMPORTANTE, para cualquiera de los cambios que haga en las políticas de red, si al aplicar alguna regla la aplicación deja de responder desde el navegador, puede eliminarla para volver a la situación anterior, e.g. *kubectl delete -f ./regla-incorrecta.yml*

3306/TCP, pero esta vez añadiremos como restricción que solo puedan acceder a él los pods con los frontales de la aplicación. Por lo tanto, además de identificar el puerto entrante, añada una restricción adicional debajo de los puertos (from.podSelector.matchLabels) y añada ahí la etiqueta que identifica a los nodos frontales. Aplique la regla y compruebe desde el navegador que la aplicación sigue funcionando. Por otro lado, si ahora trata de comprobar el acceso al puerto de la base de datos desde el pod de prueba éste debería estar cerrado.

6.- Para finalizar la segmentación inicial, debemos **restringir el tráfico de salida válido para cada tipo de pod**. Esto permitirá que si alguno de los pods resultara comprometido, no se utilizara para afectar a otros pods del clúster mediante desplazamientos laterales en la red interna o incluso evitar que pudiera conectarse a servidores externos.

- En primer lugar, **definir el tráfico saliente válido para los pods de los frontales web** modificando su regla creada en el paso 4 y añadiendo a la misma un apartado específico (*egress*.to.podSelector) dónde indiquemos como destino autorizado la etiqueta de los pods de la base de datos[13].

- Por defecto, las políticas de red de cualquier nodo permiten cualquier tipo de conexión saliente. Par evitar esto, **aplicaremos las reglas generales de denegación de conexiones entrantes y salientes** que estudiamos en el primer paso. Estas reglas generales se aplican a todos los pods de la aplicación (etiquetados como *app: asn-app*) usando un OR lógico, por lo que en principio no afectarán a las restricciones más permisivas entrantes/salientes que hemos creado para los frontales o a las reglas entrantes de la base de datos. Sin embargo, sí afectarán a las conexiones salientes de esta última que no tenía definida ninguna regla saliente específica, así como a cualquier pod futuro de la aplicación que se cree en el clúster. Como excepción a la política restrictiva de salida, debe recordarse que se habilitaron las conexiones al servicio de DNS interno del clúster para que los pods puedan conocer la dirección ip de cualquier servicio del clúster.

- Una vez aplicadas estas reglas y hayamos comprobado que la aplicación sigue respondiendo desde el navegador, si ejecutamos una shell en uno de los pods con los frontales web, veremos cómo no podemos realizar conexiones con sitios externos, aunque el servicio de DNS de k8s resuelva su ip, e.g.:

  ```
  $ kubectl exec -it wwwserver-cf6bb484b-jqhj6 -- sh
  # curl -vm 2 https://www.ujaen.es
  ```

  Como el pod de prueba no tenía la etiqueta *app: asn-app,* seguirá teniendo acceso fuera del clúster aunque, como hemos visto, ya no no puede acceder a ninguno de los pods de la aplicación.

[13] En la restricción de conexión de salida desde los frontales hacia la base de datos no es necesario indicar el puerto de destino ya que la política de entrada de la misma solo admite conexiones en dicho puerto y, por lo tanto, sería redundante.

Tarea 2.- Despliegue en clúster K8s gestionado en la nube

Una vez que hayamos establecido la segmentación de nuestra red interna[14], en este apartado procederemos a desplegar el proyecto realizado en la tarea anterior en el servicio GKE de GCP aportando las evidencias correspondientes, e.g: capturas de pantalla de las vistas de cargas de trabajo y servicios en la consola de GKE, captura de pantalla de la aplicación web respondiendo desde el navegador, etc.

Importante: consideraciones para el despliegue en GKE:

- En primer lugar, **crear un clúster estándar K8s en GKE** siguiendo las indicaciones que aparecen en el apéndice de las diapositivas del tema 4.
- Para comunicarnos con el clúster remoto usando *kubectl* desde nuestro equipo es necesario seleccionar el clúster k8s creado en GKE. Seguir las indicaciones en las diapositivas del apéndice del tema 4 sobre cómo inicializar la conexión con GKE y usar el comando "*kubectl config use-context*"[15] para interaccionar con el clúster local o con el remoto respectivamente.
- A diferencia de un despliegue local, **las imágenes de los pods deben obtenerse siempre de un registro**, e.g. docker hub, puesto que el clúster k8s en GKE no tiene acceso al repositorio de imágenes de nuestro equipo local. Por este motivo, para este despliegue la imagen de la aplicación web debe estar disponible previamente en docker hub o artifact registry y referenciarse en el fichero de despliegue debidamente, e.g. si utilizamos imágenes públicas en docker-hub las imágenes se configurarían como *image: usuario/phpwebapp* o usuario/phpwebapp:v1 dependiendo de la cuenta de usuario que se utilice en el servicio de docker y *nombre:versión* de la imagen desplegada el repositorio de docker hub.
- Debido a las **limitaciones de cpu y memoria** del único nodo utilizado en nuestro cluster GKE, es posible que al lanzar todos los pods del despliegue algunos no dispongan de recursos suficientes para su ejecución. Si esto ocurriera y algunos pods no llegaran a lanzarse en GKE, limitar en sus ficheros de despliegue el número de unidades de Memoria/CPU máximo (limit) e inicial (request) de cada pod tal y como se trabajó en la tarea 3 de la actividad 4.1[16].
- **Como GKE ya dispone de un controlador ingress integrado,** no es necesario instalar ninguno en el clúster remoto, como hicimos con traefik en la tarea 1, lo que permitirá ahorrar memoria en el clúster. Por lo tanto, solo desplegaremos la configuración del recurso ingress utilizado en la tarea anterior.
- Al crear o modificar el recurso ingress para el despliegue en GKE es normal que este **tarde incluso de 2 a 5 minutos en estar disponible** puesto que debe habilitar el servicio de balanceo de carga (*LoadBalancer*) de GCP para asignar una ip pública entrante al recurso Ingress definido en el despliegue. El estado de disponibilidad y la

[14] Si no se consigue que la segmentación de red resulte operativa y que la aplicación siga respondiendo adecuadamente tras aplicarla, se puede realizar esta segunda tarea sin llegar a desplegar las políticas de red en el clúster remoto (indicarlo en el informe).
[15] k9s permite cambiar de contexto para conectarnos a otro clúster usando el comando :contexts
[16] Una opción podría ser asignar a los pods frontales y al pod mariadb unos límites máximos e iniciales de 100m/50m (0.1/0.05) y 150m/50m (0.15/0.05) respectivamente. De igual forma se podría limitar también la memoria de cada pod si fuera necesario.

dirección IP definitiva de acceso a la aplicación se pueden consultar desde la consola de GKE, en el apartado "*Puertas de enlace, servicios & ingress → Ingress*" o lanzando desde la consola el comando "*kubectl get ingress --watch*" hasta que se muestre la ip. Mientras tanto, se puede acceder a los servicios remotos de forma directa estableciendo un túnel desde la máquina local y accediendo en la url http://localhost:8080:

```
kubectl port-forward service/wwwserver 8080:80
```

-

Para la **entrega de la actividad** se proporcionará:

- Un **fichero comprimido con la carpeta de proyecto** para poder lanzar el despliegue local, que contendrán únicamente los ficheros necesarios para ello como: Dockerfile que sean necesarios, ficheros .yaml con definición de recursos k8s, fuentes de la aplicación desplegada y ficheros de configuración adicionales necesarios para personalizar y lanzar los diferentes microservicios diseñados.
- **Informe** con un resumen de los pasos realizados y capturas de pantalla donde muestre la consecución y correcto funcionamiento de las tareas planteadas: despliegue local y despliegue en clúster GKE. Para la tarea 2 se proporcionará también la **URL de acceso al servicio desplegado en GKE** que se deberá mantener activo hasta que el profesor indique que lo ha revisado.

Extensión aproximada del informe: 4-5 páginas (principalmente por las capturas de pantalla)
Tiempo estimado: 2-3 horas (considerando la consulta de información y pruebas)
(**plazo de entrega: 13 de noviembre**)

Material de trabajo

Para la **tarea 1** se recomienda revisar los ejemplos de despliegue y servicio declarativos disponibles en las diapositivas del tema 4, además de revisar los ejemplos de *deployment* y *service* de la aplicación *testapp* y el fichero de configuración del recurso ingress, lb-ingress.yml comentados en el apartado de preparación.

[2] The NetworkPolicy resource, in *Kubernetes Documentation*. Google, 2025. [on-line] Disponible en https://kubernetes.io/docs/concepts/services-networking

[3] Balkan, A.. Kubernetes Network Policy Recipes. Google, 2017. [on-line] Disponible en https://github.com/ahmetb/kubernetes-network-policy-recipes

Para la **tarea 2** se pueden consultar las diapositivas sobre despliegue de una aplicación en GKE disponibles en los apéndices de las diapositivas del tema 4.

También se pueden consultar más detalles en las siguientes referencias:

[4] Crear un libro de visitas con Redis y Php, en Google Kubernetes Engine. Google Cloud, 2026 [on-line]. Disponible en https://cloud.google.com/kubernetes-engine/docs/tutorials/guestbook

- Útil para familiarizarse con los detalles de configuración de *deployments* y *servicios*

[5] Guía de inicio rápido, en *Google Kubernetes Engine*. Google Cloud, 2026 [on-line]. Disponible en https://cloud.google.com/kubernetes-engine/docs/quickstart?hl=es-419

- Útil para primeros pasos en creación de un clúster

[6] *Setting up HTTP loadbalancing with ingress*. Google Cloud, 2026 [on-line]. Disponible en https://cloud.google.com/kubernetes-engine/docs/tutorials/http-balancer